

BEAUTYFOLIO · PHASE 3F.4A

Backend Portability + Migration Readiness Audit

Read-only audit answering: could BeautyFolio's backend be accurately recreated from the repository if Lovable Cloud became unavailable? No fixes applied, no migration executed, no data modified.

Scope	Data modified	Backend
Read-only audit	None	Lovable Cloud (ivbujlyilzmlublqzalu)

Critical final question: "If a brand-new Supabase project were created today and the repository replayed as-is, would it reproduce the current backend?"

Answer: YES, WITH SPECIFIC GAPS — schema/RLS/functions/triggers/enums are fully reproducible with zero gaps. Real data, storage binaries, 3 absolute-URL columns, existing user passwords, and the service-role secret would each need a separate, documented step (see §24 below).

26

TABLES AUDITED

1

DRIFT FINDING
(HARMLESS)

9G / 3A / 0R

RISK AREAS

0

FIXES APPLIED (AUDIT ONLY)

1–2. Executive Verdict & Current Architecture

BeautyFolio runs on **Lovable Cloud**, Lovable's managed Supabase backend — a genuine, standard Postgres/Supabase instance accessed exclusively through the plain `@supabase/supabase-js` client. No proprietary Lovable database/auth/storage SDK exists anywhere in application code.

3–4. Live DB vs. Migration Files — Schema Drift

One real finding, self-corrected: `services` and `packages` already had a full `UNIQUE (beautician_profile_id, slug)` constraint since the very first migration — my own Phase 3F.4 audit missed this (inline in a `CREATE TABLE` block, didn't match my search pattern). My Phase 3F.4 migration added a second, functionally redundant partial index. Harmless — both now coexist, no conflict, just duplication. Documented, not fixed here per the audit-only instruction.

No other drift found across 26 tables, 11 functions, 18 triggers, 37 RLS policies, 11 enums — every live object traces to a migration file.

5. Migration-History Findings

Lovable Cloud doesn't expose the standard Supabase CLI migration-history table from this access path. Reconciled instead via exhaustive direct schema comparison (§3–4) — found fully consistent.

6–9. Tables, RLS, RPC, Triggers

Category	Count	Finding
Tables	26	All in <code>public</code> , RLS enabled on every one, no views/sequences
RLS policies	37	All trace to shared helpers <code>owns_beautician_profile()</code> / <code>is_published_profile()</code>
Functions/RPCs	11	All present in migrations; <code>submit_lead()</code> + <code>log_admin_action()</code> actively used by app code
Triggers	18	15 <code>updated_at</code> + 2 write-guards + 1 signup-automation, all traced

`portfolio_events` table + `record_portfolio_event()` RPC exist in both migrations and live DB but are **never called by any application code** — dormant analytics infrastructure, not drift.

10–11. Auth Architecture & Migration Risk

- Email/password only — no OAuth, no magic link found in the app
- `handle_new_user()` trigger auto-creates `profiles` + default `beautician` role on signup
- Role/ownership checks: `user_roles` table + `has_role()` / `owns_beautician_profile()` — no metadata-based role logic

Real risk found: `profiles.auth_user_id` has **no database-level foreign key** to `auth.users.id` — kept in sync only by the signup trigger. A future Auth migration needs explicit remapping. Supabase also doesn't support standard password-hash export — existing users would need a reset-password flow post-migration.

12–14. Storage Inventory, Migration Risk & URL Dependencies

One bucket: `portfolio-media` , public, 4 RLS policies, fully reproducible from migrations. Database backup alone would **not** recreate the actual files — needs a separate object-copy procedure (documented 8-step plan in the full doc).

Storage-referencing column	Classification
Gallery / Before-After / uploaded-video <code>storage_path</code>	PORTABLE — relative path, resolved dynamically
<code>portfolio_videos.video_url</code> (YouTube/Instagram)	PORTABLE — external URL, not backend-specific
<code>beautician_profiles.profile_image_url</code> / <code>cover_image_url</code>	REQUIRES URL REWRITE — absolute Supabase URL baked in
<code>portfolio_videos.thumbnail_url</code>	REQUIRES URL REWRITE — same

15. Environment Variable Inventory

8 variables inventoried (names only, no values shown per instruction). **Notable finding:** `SUPABASE_SERVICE_ROLE_KEY` is injected by Lovable Cloud at runtime — absent from the repo's `.env` entirely — and powers an admin/service-role client (`supabaseAdmin`) that is **defined but imported nowhere in the codebase**. Dormant, not a current dependency of any working feature.

16. Lovable-Specific Dependencies

Dependency	Classification
@lovable.dev/vite-tanstack-config (build config package)	D — backend/build infrastructure dependency, but replaceable with standard Vite/TanStack config
window.__lovableEvents error-reporting hooks	A — development-only, no-op in any real production browser
SUPABASE_SERVICE_ROLE_KEY runtime injection	D — but currently unused by any working feature

17–19. Edge Functions, Admin System, CRM

- **No Supabase Edge Functions exist** — supabase/functions/ doesn't exist; all server logic runs through TanStack Start server functions in the one app deployment
- Admin system: pure RLS/role-based (has_role()), no Lovable-specific identity state
- CRM: full FK chain (leads → lead_inquiries → lead_inquiry_services , lead_activities) intact; lead_status enum evolution (6→9 values across two migrations) matches live exactly

20–21. Enums & Seed/Demo Data Separation

No seed/demo-data migration exists at all. Every one of the 13 migration files is schema-only — zero INSERT statements populating demo content. A clean replay of the migrations carries **zero risk** of accidentally seeding test/demo data into a future production migration, because there is no seed script to run.

22–23. Legacy Fields & Service-Slug Verification

rating / review_count / client_count / metrics_verified classification unchanged from Phase 3F.3A. Phase 3F.4's service-slug work (column, CHECK constraint, unique index, backfilled data, updateService() preservation behavior) fully re-verified live-vs-migration-consistent this phase.

24. Future Migration Procedure (Not Executed)

18-step procedure documented in the full report: freeze → export schema → provision destination → replay migrations → migrate Auth → migrate tables → migrate Storage → recreate policies/RLS/functions/triggers → configure environment → validate row/object/FK counts → regression test → cutover → monitor → rollback window.

25. Future Validation Checklist (Not Executed)

Row counts per table, published-profile counts, service/gallery/review/lead counts, Storage object count vs. reference count (orphan detection both directions), Auth user count, broken-FK scan, duplicate-slug scan, post-rewrite URL resolution check.

26. GREEN / AMBER / RED Risk Matrix

Database schema	GREEN	Migration history	AMBER	RLS	GREEN
Functions/RPC	GREEN	Triggers	GREEN	Auth	AMBER

Storage	AMBER	Environment variables	GREEN	Lovable-specific deps	AMBER
Application code	GREEN	SEO infrastructure	GREEN	CRM data	GREEN

No meaningless single overall percentage given, per instruction — this matrix is the answer.

27–30. Documentation, Gaps, Manual Actions & Remediation

- **Created:** docs/architecture/backend-portability.md (full findings) + backend-portability-manifest.json (machine-readable)
- **Missing from migrations:** nothing schema-relevant — only my own prior audit's oversight (§3–4), now corrected in documentation
- **Manual actions needed from you:** none right now — only relevant if a real migration is later approved (destination secrets, password-reset communication plan)
- **Recommended remediation (not started):** (1) drop the one redundant index, (2) build a ready-to-run storage-URL-rewrite script, (3) decide the fate of the dormant service-role client